# NVIDIA Warp Migration Plan

P(F) Shape Matching PBD -> Differentiable Simulation

*C++/CUDA (forward-only) -> Python/Warp (auto-differentiable)*

## 1. Environment Verification

Warp 1.10.1 installed, CUDA 12.8, RTX 2070 (sm_75). Key APIs confirmed:

| Warp API | Status | CUDA Equivalent |
|---|---|---|
| wp.svd3 | YES | svd3_cuda.h (custom header) |
| wp.atomic_add | YES | atomicAdd() |
| wp.HashGrid | YES | CUB sort + cell_start/end |
| wp.Tape | YES | N/A (no autodiff in CUDA) |
| wp.vec3 / wp.mat33 | YES | float3 / manual 3x3 |
| wp.kernel | YES | __global__ void |

**All blockers cleared. Full migration is feasible.**

## 2. Migration Scope: 12 CUDA Kernels -> Warp Kernels

| CUDA Kernel | Lines | Difficulty | Warp Notes |
|---|---|---|---|
| predict_kernel | 15 | Easy | vel += g*dt, pred = pos + vel*dt |
| collision_kernel | 25 | Easy | floor clamp + sphere SDF |
| clear_deltas_kernel | 5 | Trivial | wp.zeros() |
| cluster_solve_kernel | 90 | Hard | wp.svd3 + wp.atomic_add |
| apply_kernel | 10 | Easy | pred += stiff * delta / count |
| update_velocity_kernel | 10 | Easy | vel = (pred - old) / dt |
| update_edge_damage | 30 | Medium | sigmoid P(F) + max damage |
| update_particle_damage | 15 | Medium | wp.atomic_max (custom) |
| aspirate_kernel | 10 | Easy | soft mask for differentiable |
| compute_cell_hash | 15 | Easy | wp.HashGrid handles this |
| find_cell_bounds | 10 | N/A | wp.HashGrid built-in |
| self_collision_kernel | 60 | Medium | wp.hash_grid_query() |

**Core challenge:** cluster_solve_kernel (warp shuffle reduction + SVD). Warp has no warp shuffle, need to restructure as per-particle kernel with shared memory or sequential accumulation per cluster.

## 3. Warp Architecture Design

### 3.1 Data Layout

```
# Particle data (Structure of Arrays)
pos       = wp.zeros(N, dtype=wp.vec3, device="cuda:0")
vel       = wp.zeros(N, dtype=wp.vec3, device="cuda:0")
pred_pos  = wp.zeros(N, dtype=wp.vec3, device="cuda:0")
old_pos   = wp.zeros(N, dtype=wp.vec3, device="cuda:0")
damage    = wp.zeros(N, dtype=wp.float32, device="cuda:0")
removed   = wp.zeros(N, dtype=wp.float32, device="cuda:0")  # float for differentiable
fixed     = wp.zeros(N, dtype=wp.int32, device="cuda:0")

# Cluster data (CSR format, same as CUDA)
cluster_starts  = wp.array(starts_np, dtype=wp.int32, device="cuda:0")
cluster_sizes   = wp.array(sizes_np, dtype=wp.int32, device="cuda:0")
particle_ids    = wp.array(ids_np, dtype=wp.int32, device="cuda:0")
rest_offsets    = wp.array(offsets_np, dtype=wp.vec3, device="cuda:0")

# Delta buffers
deltas       = wp.zeros(N, dtype=wp.vec3, device="cuda:0")
delta_counts = wp.zeros(N, dtype=wp.int32, device="cuda:0")
```

### 3.2 Key Differentiability Change: Soft Removal

```
# CUDA (non-differentiable):
if damage[i] > 0.3:
    removed[i] = 1       # hard binary decision

# Warp (differentiable):
alpha[i] = wp.sigmoid(-k * (damage[i] - threshold))  # soft mask [0,1]
# alpha ~ 1 when healthy, alpha ~ 0 when damaged
# All subsequent computations use alpha[i] as weight:
#   pred_pos[i] contributes alpha[i] to cluster solve
#   mass_effective[i] = alpha[i] * mass[i]
```

### 3.3 Cluster Solve Restructuring

CUDA uses 1 warp (32 threads) per cluster with __shfl_sync reduction. Warp has no warp shuffle. Two options:

**Option A (recommended):** Per-cluster sequential loop in kernel. Each kernel instance handles one cluster, loops over its particles sequentially. Simple, correct, differentiable. Slower than warp shuffle but adequate for <30k clusters.

**Option B:** Two-pass: Pass 1 = per-particle kernel computes partial sums (cm, A_pq contributions), atomicAdd to per-cluster accumulators. Pass 2 = per-cluster kernel does SVD + goal positions. More parallel but more complex.

```
# Option A: Per-cluster kernel (recommended for Warp)
@wp.kernel
def cluster_solve_kernel(
    pred_pos: wp.array(dtype=wp.vec3),
    alpha: wp.array(dtype=wp.float32),     # soft removal mask
    cluster_starts: wp.array(dtype=wp.int32),
    cluster_sizes: wp.array(dtype=wp.int32),
    particle_ids: wp.array(dtype=wp.int32),
    rest_offsets: wp.array(dtype=wp.vec3),
    deltas: wp.array(dtype=wp.vec3),
    delta_counts: wp.array(dtype=wp.int32)):

    cid = wp.tid()
    start = cluster_starts[cid]
    size = cluster_sizes[cid]

    # 1. Weighted centroid (sequential loop)
    cm = wp.vec3(0.0, 0.0, 0.0)
    total_w = float(0.0)
    for j in range(size):
        pid = particle_ids[start + j]
        w = alpha[pid]
        cm += w * pred_pos[pid]
        total_w += w
    if total_w < 1.5: return
    cm = cm / total_w

    # 2. Covariance matrix A_pq (sequential)
    # ... similar loop building 3x3 matrix ...

    # 3. SVD
    U, sigma, V = wp.svd3(A)
    R = U * wp.transpose(V)

    # 4. Goal positions -> atomic_add
    for j in range(size):
        pid = particle_ids[start + j]
        if alpha[pid] < 0.01: continue
        q = rest_offsets[start + j]
        goal = R * q + cm
        correction = goal - pred_pos[pid]
        wp.atomic_add(deltas, pid, correction)
        wp.atomic_add(delta_counts, pid, 1)
```

## 4. 6-Phase Implementation Plan

### Phase 1: Skeleton + Mesh Loading (Week 1)

1. Create project: pf_shape_matching_warp/
2. Use meshio to load .msh tet mesh -> numpy arrays
3. Build cluster data from tet adjacency (reuse algorithm from tet_mesh.h)
4. Initialize Warp arrays (pos, vel, clusters CSR, edges)
5. Verify: print particle count, cluster count, edge count

```
import meshio, numpy as np, warp as wp

mesh = meshio.read("cube32.msh")
vertices = mesh.points          # (N, 3)
tets = mesh.cells_dict["tetra"] # (T, 4)

wp.init()
pos = wp.array(vertices, dtype=wp.vec3, device="cuda:0")
```

**Deliverable: Warp arrays initialized from .msh file, cluster CSR built.**

### Phase 2: Core Shape Matching (Week 2-3)

1. Implement predict_kernel (trivial)
2. Implement collision_kernel (floor + knife sphere SDF)
3. Implement cluster_solve_kernel with wp.svd3 (core challenge)
4. Implement apply_kernel, clear_deltas, update_velocity
5. Verify: cube drops onto floor, deforms, settles. Compare with C++ output.

```
# Validation: compare Warp vs CUDA output
# Run both 100 steps on same input, compare final positions
max_diff = np.max(np.abs(warp_pos - cuda_pos))
assert max_diff < 1e-3, f"Mismatch: {max_diff}"
```

**Deliverable: Cube falls onto floor correctly. Numerically matches CUDA within 1e-3.**

### Phase 3: P(F) Damage + Soft Removal (Week 4)

1. Implement P(F) edge damage kernel (sigmoid, knife surface distance)
2. Implement particle damage aggregation (atomic max via atomic_add trick)
3. Implement SOFT removal: alpha[i] = sigmoid(-k*(damage-threshold))
4. Modify cluster_solve to weight by alpha[i]
5. Verify: knife contact -> damage accumulates -> particles fade out

**Deliverable: Debulking works with soft differentiable removal.**

### Phase 4: Autodiff Verification (Week 5)

1. Wrap simulation loop in wp.Tape()
2. Define loss: L = -V_removed + lambda * nerve_damage
3. Call tape.backward(loss)
4. Verify: dL/d(tool_pos) has correct sign (moving toward tumor -> gradient negative)
5. Finite difference validation: compare analytic gradient vs FD

```
tape = wp.Tape()
with tape:
    for step in range(100):
        simulate_step(tool_pos, ...)
    loss = compute_loss(damage, alpha, nerve_mask)

tape.backward(loss)
grad_tool = tape.gradients[tool_pos]  # dL/d(tool_pos)

# Finite difference check
eps = 1e-4
tool_pos_plus = tool_pos + eps
loss_plus = run_forward(tool_pos_plus)
grad_fd = (loss_plus - loss) / eps
assert abs(grad_tool - grad_fd) / abs(grad_fd) < 0.05  # <5% error
```

**Deliverable: Analytic gradients match finite difference within 5%.**

### Phase 5: Trajectory Optimization (Week 6-7)

1. Parameterize tool trajectory: tau(t) = spline with N control points
2. Objective: max tumor_removed - lambda * nerve_damage
3. Optimizer: Adam on control points, using Warp gradients
4. Baseline comparisons: random, PPO, CEM
5. Generate figures: optimized trajectory visualization, Pareto curve

```
control_points = wp.array(init_trajectory, dtype=wp.vec3, requires_grad=True)
optimizer = torch.optim.Adam([control_points.numpy()], lr=1e-3)

for epoch in range(200):
    tape = wp.Tape()
    with tape:
        trajectory = interpolate_spline(control_points, T)
        for t in range(T):
            set_tool_pos(trajectory[t])
            simulate_step(...)
        loss = -volume_removed + lambda_val * nerve_damage
    tape.backward(loss)
    optimizer.step()
```

**Deliverable: Optimized trajectory avoids nerve, removes max tumor. Paper-ready figures.**

### Phase 6: Visualization + Paper Experiments (Week 8)

1. Polyscope or Open3D visualization of trajectory + damage field
2. Ablation: P(F) vs threshold, differentiable vs non-differentiable
3. Scalability: 5k, 20k, 100k particles timing
4. Heterogeneous tissue: 3-layer nerve preservation demo
5. Paper writing: CoRL / ICRA format

**Deliverable: Complete paper draft with all figures and experiments.**

## 5. Warp Project File Structure

```
pf_shape_matching_warp/
  |-- sim/
  |   |-- particles.py        # Particle data + wp.array initialization
  |   |-- clusters.py         # Cluster building from tet mesh + CSR packing
  |   |-- kernels.py          # All @wp.kernel definitions (predict, solve, apply...)
  |   |-- pf_damage.py        # P(F) edge damage + soft removal kernels
  |   |-- solver.py           # ShapeMatchingSolver class (step, init, loss)
  |
  |-- mesh/
  |   |-- loader.py           # meshio .msh loader -> numpy arrays
  |   |-- heterogeneous.py    # 3-layer tissue model
  |
  |-- optim/
  |   |-- trajectory.py       # Spline trajectory parameterization
  |   |-- optimizer.py        # Adam/L-BFGS on Warp gradients
  |   |-- baselines.py        # PPO, CEM, random baselines
  |
  |-- viz/
  |   |-- renderer.py         # Polyscope / Open3D visualization
  |
  |-- experiments/
  |   |-- exp_gradient_check.py   # Finite difference validation
  |   |-- exp_trajectory_opt.py   # Task 1: trajectory optimization
  |   |-- exp_pareto.py           # Task 3: volume vs nerve Pareto
  |   |-- exp_ablation.py         # P(F) vs threshold ablation
  |
  |-- main.py                 # Entry point
```

## 6. CUDA -> Warp API Mapping Reference

| CUDA | Warp |
|---|---|
| __global__ void kernel(...) | @wp.kernel def kernel(...) |
| kernel<<<blocks, threads>>>(...) | wp.launch(kernel, dim=N, ...) |
| float3 | wp.vec3 |
| float3* d_array | wp.array(dtype=wp.vec3) |
| cudaMalloc / cudaFree | wp.zeros() / automatic GC |
| cudaMemcpy H->D | wp.array(numpy_arr, device="cuda:0") |
| cudaMemcpy D->H | array.numpy() |
| atomicAdd(&x, val) | wp.atomic_add(array, idx, val) |
| __shfl_down_sync() | sequential loop (no warp shuffle) |
| svd3_cuda.h svd() | wp.svd3(A) |
| CUB radix sort + cell arrays | wp.HashGrid |
| N/A (no autodiff) | wp.Tape() + tape.backward() |
| cudaDeviceSynchronize() | wp.synchronize() |

## 7. Risks and Mitigations

| Risk | Impact | Mitigation | Timeline Impact |
|---|---|---|---|
| cluster_solve perf | Medium | Sequential loop OK for <30k | +0 (acceptable) |

| | | | |
|---|---|---|---|
| wp.svd3 gradient | High | Test early in Phase 2 | +1 week if fails |
| Tape memory for | Medium | Checkpoint every N steps | +2 days |
| long rollouts | | | |
| HashGrid differentiable? | Low | Self-collision not critical | +0 |
| meshio .msh v4.1 | Low | Already tested with meshio | +0 |

## 8. Timeline Summary

| Week | Task | Deliverable |
|---|---|---|
| 1 | Mesh loading + Warp array init | Particles + clusters in Warp |
| 2-3 | Core shape matching (SVD) | Cube falls correctly, matches CUDA |
| 4 | P(F) damage + soft removal | Debulking with differentiable mask |
| 5 | Autodiff verification | Gradients match FD within 5% |
| 6-7 | Trajectory optimization | Nerve-avoiding path, Pareto curves |
| 8 | Visualization + paper | Complete draft + figures |

**Total: 8 weeks. Critical path: Week 2-3 (SVD kernel) -> Week 5 (gradient check).**

## 9. Success Criteria

1. Shape matching simulation matches CUDA output within 1e-3 (100 steps)
2. P(F) damage accumulates correctly with soft sigmoid mask
3. Analytic gradients (via wp.Tape) match finite difference within 5%
4. Trajectory optimization converges: nerve_damage decreases, volume_removed increases
5. Analytic gradient method outperforms PPO baseline in sample efficiency (100x fewer rollouts)
6. 20k particles runs at >30 FPS forward, backward pass <100ms